Scholay学术 软件设计说明书

软件名称： Scholay学术

版本号： V1.0

著作权人： 成都恩特罐罐文化科技有限公司

统一社会信用代码： 91510100MAEEHRNE26

编制日期： 2025年1月

版权声明

Copyright © 2025 成都恩特罐罐文化科技有限公司

All Rights Reserved. 保留所有权利。

目录

1. 引言
2. 系统概述
3. 系统架构设计
4. 数据库设计
5. 功能模块设计
6. 接口设计
7. 安全设计
8. 部署设计
9. 测试说明
10. 附录

第一章 引言

1.1 编写目的

本文档是Scholay学术搜索平台系统的软件设计说明书，旨在详细描述系统的整体架构、功能模块、数据库设计、接口规范等技术细节。本文档可作为软件开发、测试、部署和维护的技术依据，同时也是软件著作权登记的技术文档材料。

本文档的主要读者包括：

- 软件开发工程师：了解系统架构和实现细节
- 测试工程师：制定测试计划和测试用例
- 运维工程师：了解部署架构和运维要求
- 项目管理人员：掌握系统整体技术方案

1.2 项目背景

随着科学研究的快速发展，学术论文数量呈爆发式增长。研究人员面临着海量文献检索困难、论文分析耗时等问题。传统的学术搜索引擎功能单一，无法满足现代科研工作者的多元化需求。

Scholay学术搜索平台是一款面向科研人员的智能化学术论文检索与AI辅助分析系统。系统整合了Semantic Scholay学术数据库，结合先进的人工智能技术，为用户提供高效、智能的学术研究支持服务。

1.3 定义与缩略语

术语	定义
API	Application Programming Interface，应用程序接口
JWT	JSON Web Token，用于身份认证的令牌
REST	Representational State Transfer，表现层状态转移
SSE	Server-Sent Events，服务器推送事件
MCP	Model Context Protocol，模型上下文协议
GORM	Go语言的对象关系映射库
Redis	高性能的键值存储数据库
Docker	容器化部署平台
Nginx	高性能HTTP服务器和反向代理
OAuth	Open Authorization，开放授权协议

1.4 参考资料

1. Semantic Scholar API Documentation
2. Go语言官方文档 (golang.org)
3. React 18官方文档 (react.dev)
4. MySQL 8.0参考手册
5. Redis官方文档
6. Docker官方文档
7. Nginx官方文档
8. JSON Web Token (JWT) RFC 7519

第二章 系统概述

2.1 系统目标

Scholay学术搜索平台的主要目标是：

1. **高效检索**：提供快速、准确的学术论文检索服务，支持多种搜索模式
2. **智能分析**：利用AI技术对论文进行深度分析，提取关键信息
3. **个性化服务**：根据用户研究领域提供定制化推荐
4. **便捷交互**：提供直观友好的用户界面，降低使用门槛

5. **稳定可靠**：保证系统高可用性和数据安全性

2.2 系统功能概述

系统主要包含以下核心功能：

2.2.1 用户管理功能

- 用户注册与登录（邮箱、手机号、OAuth）
- 个人信息管理
- 会话管理与安全控制

2.2.2 学术搜索功能

- 简单搜索：基于关键词的快速检索
- 高级搜索：支持多条件组合查询
- 智能搜索：AI辅助的语义理解搜索

2.2.3 论文分析功能

- 论文详情展示
- AI智能摘要生成
- 引用关系分析
- 作者学术画像

2.2.4 AI对话功能

- 会话式论文问答
- 多轮对话支持
- 上下文理解

2.2.5 订阅管理功能

- 订阅计划展示
- 订单创建与支付
- 配额管理

2.2.6 系统管理功能

- 用户管理
- 数据统计
- 系统配置

2.3 用户角色

系统定义以下用户角色：

角色	描述	主要权限
游客	未登录用户	浏览公开内容

角色	描述	主要权限
普通用户	已注册的免费用户	基础搜索、有限次数的AI功能
付费用户	订阅付费计划的用户	高级搜索、无限AI功能
管理员	系统管理人员	用户管理、系统配置、数据统计

2.4 运行环境

2.4.1 硬件环境

服务器配置要求：

- CPU：4核心及以上
- 内存：8GB及以上
- 存储：100GB SSD及以上
- 网络：100Mbps及以上带宽

2.4.2 软件环境

服务端：

- 操作系统：Linux (Ubuntu 20.04/22.04 LTS)
- 容器平台：Docker 24.0+
- 编排工具：Docker Compose 2.0+
- 数据库：MySQL 8.0
- 缓存：Redis 7.0
- Web服务器：Nginx 1.25

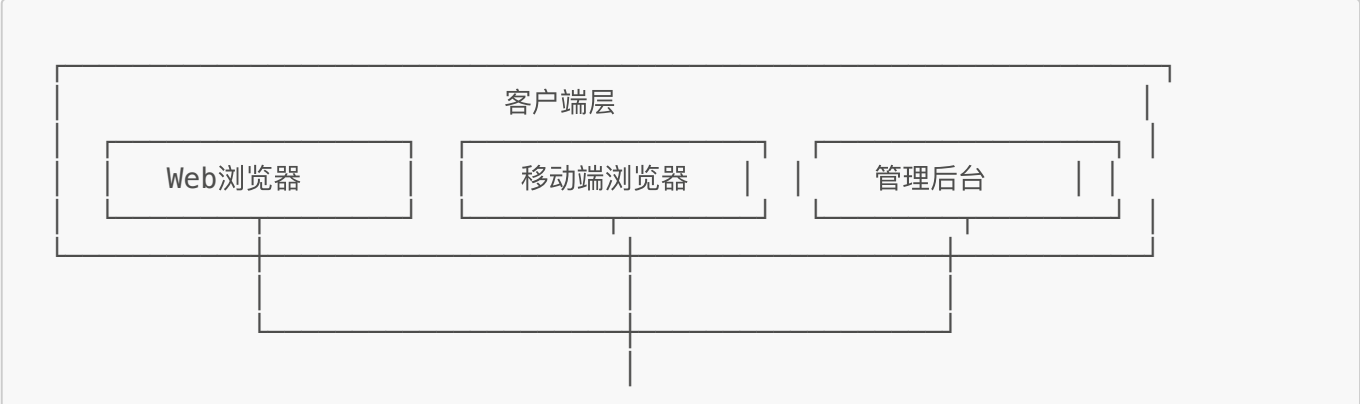
客户端：

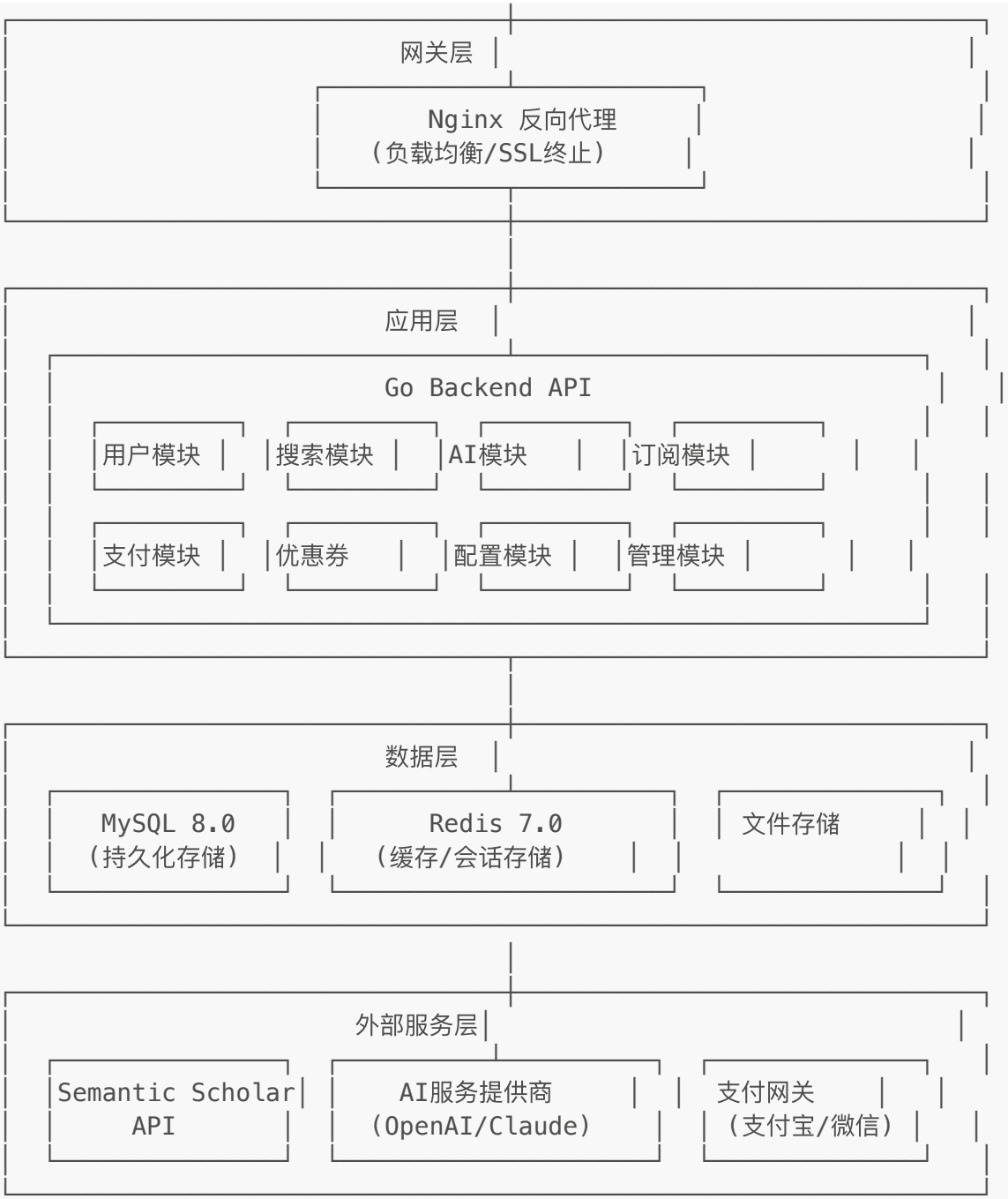
- 浏览器：Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- 支持移动端浏览器访问

第三章 系统架构设计

3.1 总体架构

Scholay学术搜索平台采用前后端分离的B/S架构，系统整体架构如下：





3.2 技术架构

3.2.1 前端技术栈

用户门户（frontend-user）：

- 框架：React 18
- 路由：React Router v6
- HTTP客户端：Axios
- PDF预览：react-pdf + pdfjs-dist
- 图表：Recharts
- Markdown渲染：react-markdown + rehype-highlight
- 验证组件：rc-slider-captcha
- 构建工具：Create React App

管理后台（frontend-admin）：

- 框架：React 18
- 路由：React Router v6
- HTTP客户端：Axios
- 图表：Recharts
- UI组件：自定义组件库
- 构建工具：Create React App

3.2.2 后端技术栈

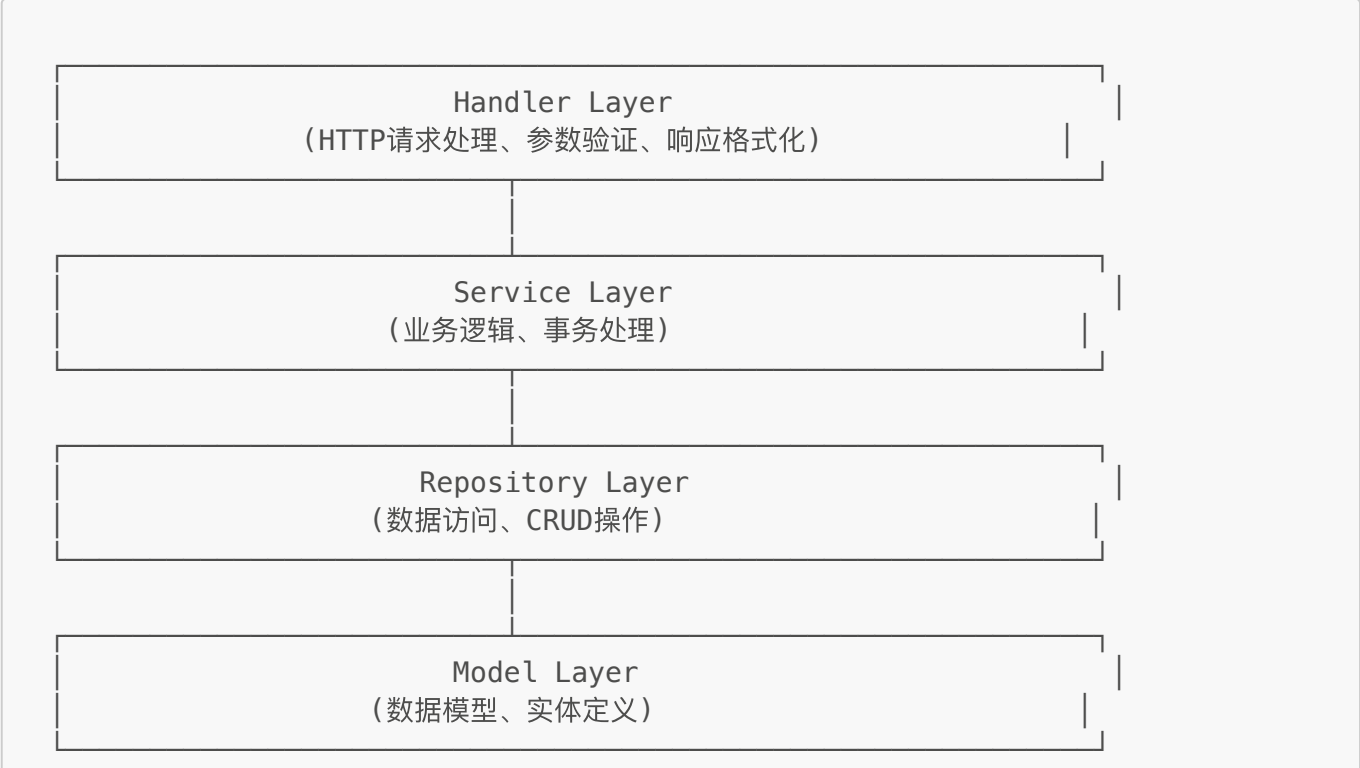
- 语言：Go 1.21
- Web框架：Gin
- ORM：GORM
- 数据库：MySQL 8.0
- 缓存：Redis 7.0
- 认证：JWT v5
- 熔断器：gobreaker
- 配置：godotenv

3.2.3 基础设施

- 容器化：Docker
- 编排：Docker Compose
- 反向代理：Nginx
- 进程管理：Supervisor（可选）

3.3 后端分层架构

后端采用清晰的分层架构设计：



3.4 模块划分

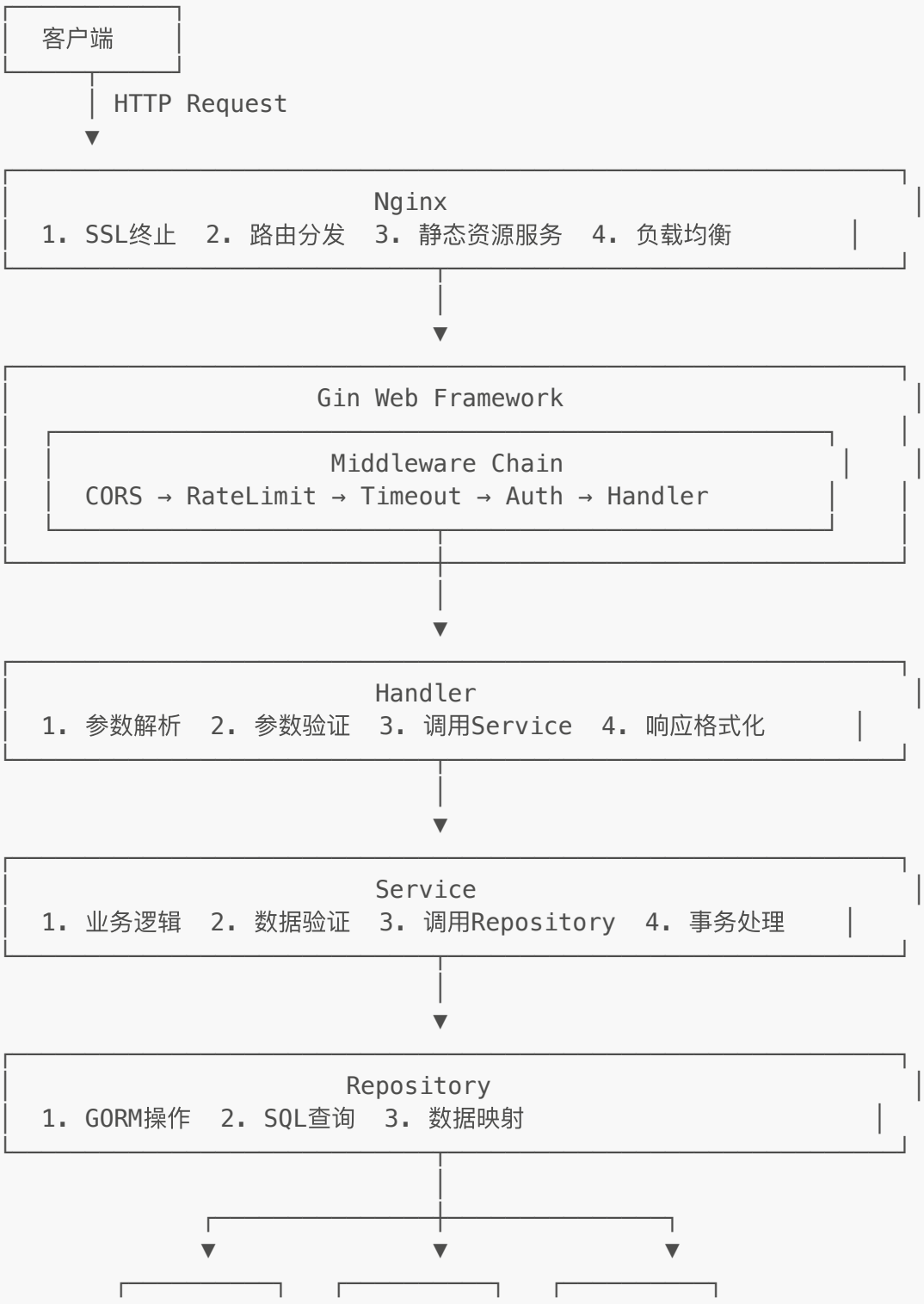
系统后端按业务领域划分为以下模块：

```
backend/internal/
├── admin/              # 管理员模块
│   ├── handler.go      # HTTP处理器
│   ├── service.go      # 业务逻辑
│   ├── repository.go   # 数据访问
│   └── model.go        # 数据模型
├── user/               # 用户模块
│   ├── handler.go
│   ├── service.go
│   ├── repository.go
│   └── model.go
├── ai/                 # AI服务模块
│   ├── handler.go
│   ├── service.go
│   └── prompt.go       # 提示词模板
├── conversation/       # 对话管理模块
│   ├── handler.go
│   ├── service.go
│   └── repository.go
├── semanticscholar/    # 学术搜索模块
│   ├── handler.go
│   ├── service.go
│   └── client.go       # API客户端
├── subscription/       # 订阅管理模块
│   ├── handler.go
│   ├── service.go
│   └── repository.go
├── order/              # 订单管理模块
│   ├── handler.go
│   ├── service.go
│   └── repository.go
├── payment/            # 支付模块
│   ├── alipay_*.go     # 支付宝集成
│   └── wechat_pay_*.go # 微信支付集成
├── coupon/             # 优惠券模块
│   ├── handler.go
│   ├── service.go
│   └── repository.go
├── config/             # 系统配置模块
│   ├── handler.go
│   └── service.go
├── proxy/              # HTTP代理模块
│   └── handler.go
├── mcp/                # MCP协议模块
│   └── handler.go
└── common/             # 公共模块
    ├── config/         # 应用配置
```

```
├── database/        # 数据库连接
├── middleware/      # HTTP中间件
├── cache/           # Redis缓存
├── circuitbreaker/  # 熔断器
└── utils/           # 工具函数
```

3.5 请求处理流程

用户请求的处理流程如下：



```
        │  MySQL   │    │  Redis   │    │ 外部API  │
        └──────────┘    └──────────┘    └──────────┘
```

第四章 数据库设计

4.1 数据库概述

系统使用MySQL 8.0作为主数据库，采用InnoDB存储引擎，支持事务和外键约束。数据库字符集统一使用utf8mb4，支持完整的Unicode字符（包括emoji表情）。

4.2 数据库表清单

系统共包含31张数据表，按功能模块分类如下：

模块	表名	说明
核心用户	admin_users	管理员用户表
核心用户	users	用户表
核心用户	user_auth_providers	用户认证提供者表
会话统计	user_sessions	用户会话表
会话统计	user_statistics	用户统计表
会话统计	online_user_snapshots	在线用户快照表
对话消息	conversations	对话表
对话消息	conversation_messages	对话消息表
订阅配额	subscription_plans	订阅计划表
订阅配额	user_subscriptions	用户订阅表
订阅配额	usage_quotas	使用配额表
订单支付	orders	订单表
订单支付	payment_transactions	支付交易表
优惠券	coupon_templates	优惠券模板表
优惠券	user_coupons	用户优惠券表
优惠券	coupon_usage_logs	优惠券使用记录表
优惠券	user_invitations	用户邀请表
优惠券	invitation_relationships	邀请关系表
优惠券	coupon_batches	优惠券批次表
系统配置	third_party_accounts	第三方账户表

模块	表名	说明
系统配置	prompt_templates	提示词模板表
系统配置	system_configs	系统配置表
系统配置	github_oauth_config	GitHub OAuth配置表
系统配置	wechat_configs	微信小程序配置表
系统配置	wechat_official_config	微信公众号配置表
系统配置	wechat_pay_configs	微信支付配置表
系统配置	alipay_configs	支付宝配置表
系统配置	email_configs	邮件配置表
系统配置	sms_configs	短信配置表
系统配置	privacy_policies	隐私政策表
系统配置	terms_of_service	服务条款表

4.3 核心表结构设计

4.3.1 管理员用户表 (admin_users)

管理员用户表存储后台管理员的账户信息。

```
CREATE TABLE IF NOT EXISTS `admin_users` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `username` VARCHAR(50) NOT NULL COMMENT '用户名',
  `password_hash` VARCHAR(255) NOT NULL COMMENT '密码哈希',
  `email` VARCHAR(100) COMMENT '邮箱',
  `status` TINYINT NOT NULL DEFAULT 1 COMMENT '状态: 1=激活, 0=禁用',
  `last_login_at` DATETIME COMMENT '最后登录时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_username` (`username`),
  INDEX `idx_status` (`status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='管理员用户表';
```

字段说明:

字段名	类型	约束	说明
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	主键ID

字段名	类型	约束	说明
username	VARCHAR(50)	NOT NULL, UNIQUE	登录用户名
password_hash	VARCHAR(255)	NOT NULL	密码的bcrypt哈希值
email	VARCHAR(100)	NULL	管理员邮箱
status	TINYINT	NOT NULL, DEFAULT 1	账户状态
last_login_at	DATETIME	NULL	最后登录时间
created_at	DATETIME	NOT NULL	记录创建时间
updated_at	DATETIME	NOT NULL	记录更新时间

4.3.2 用户表 (users)

用户表存储平台用户的基本信息，支持多种认证方式。

```
CREATE TABLE IF NOT EXISTS `users` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `username` VARCHAR(50) COMMENT '用户名',
  `email` VARCHAR(100) COMMENT '邮箱',
  `phone` VARCHAR(20) COMMENT '手机号',
  `password_hash` VARCHAR(255) COMMENT '密码哈希',
  `avatar_url` VARCHAR(255) COMMENT '头像URL',
  `bio` TEXT COMMENT '用户简介',
  `location` VARCHAR(100) COMMENT '用户位置',
  `website` VARCHAR(255) COMMENT '个人网站',
  `status` TINYINT NOT NULL DEFAULT 1 COMMENT '状态: 1=激活, 0=禁用, 2=暂停',
  `email_verified` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '邮箱是否验证',
  `phone_verified` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '手机是否验证',
  `subscription_plan` VARCHAR(20) NOT NULL DEFAULT 'free' COMMENT '订阅计
划',
  `subscription_expires_at` DATETIME COMMENT '订阅过期时间',
  `last_login_at` DATETIME COMMENT '最后登录时间',
  `last_login_ip` VARCHAR(45) COMMENT '最后登录IP',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_username` (`username`),
  UNIQUE INDEX `uk_email` (`email`),
  UNIQUE INDEX `uk_phone` (`phone`),
  INDEX `idx_status` (`status`),
  INDEX `idx_subscription_plan` (`subscription_plan`),
  INDEX `idx_last_login_at` (`last_login_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='用户表';
```

字段说明：

字段名	类型	约束	说明
id	BIGINT UNSIGNED	PRIMARY KEY	主键ID
username	VARCHAR(50)	UNIQUE	用户名
email	VARCHAR(100)	UNIQUE	邮箱地址
phone	VARCHAR(20)	UNIQUE	手机号码
password_hash	VARCHAR(255)	NULL	密码哈希
avatar_url	VARCHAR(255)	NULL	头像URL
bio	TEXT	NULL	用户简介
location	VARCHAR(100)	NULL	地理位置
website	VARCHAR(255)	NULL	个人网站
status	TINYINT	NOT NULL	账户状态
email_verified	BOOLEAN	NOT NULL	邮箱验证状态
phone_verified	BOOLEAN	NOT NULL	手机验证状态
subscription_plan	VARCHAR(20)	NOT NULL	订阅计划代码
subscription_expires_at	DATETIME	NULL	订阅到期时间
last_login_at	DATETIME	NULL	最后登录时间
last_login_ip	VARCHAR(45)	NULL	最后登录IP
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

4.3.3 用户认证提供者表 (user_auth_providers)

存储用户的第三方登录认证信息，支持OAuth、微信等多种认证方式。

```
CREATE TABLE IF NOT EXISTS `user_auth_providers` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
  `provider` VARCHAR(20) NOT NULL COMMENT '认证提供者: email, phone, wechat,
google, github',
  `provider_user_id` VARCHAR(255) NOT NULL COMMENT '第三方平台用户ID',
  `provider_union_id` VARCHAR(255) COMMENT '第三方平台联合ID',
  `provider_data` JSON COMMENT '第三方平台额外数据',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
```

```
  UNIQUE KEY `uk_provider_user` (`provider`, `provider_user_id`),
  UNIQUE KEY `uk_provider_union` (`provider`, `provider_union_id`),
  INDEX `idx_user_id` (`user_id`),
  CONSTRAINT `fk_user_auth_user` FOREIGN KEY (`user_id`) REFERENCES
`users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='用户认证提供者表';
```

4.3.4 对话表 (conversations)

存储用户与AI的对话会话信息。

```
CREATE TABLE IF NOT EXISTS `conversations` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
  `title` VARCHAR(255) NOT NULL COMMENT '对话标题',
  `conversation_type` VARCHAR(20) NOT NULL DEFAULT 'search' COMMENT '对话类
型: chat或search',
  `model_level` VARCHAR(20) DEFAULT 'low' COMMENT 'AI模型级别: low, mid,
high',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  INDEX `idx_user_id` (`user_id`),
  INDEX `idx_updated_at` (`updated_at`),
  INDEX `idx_conversation_type` (`conversation_type`),
  INDEX `idx_user_type_updated` (`user_id`, `conversation_type`,
`updated_at` DESC),
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='用户对话表';
```

4.3.5 对话消息表 (conversation_messages)

存储对话中的具体消息内容，支持消息链结构。

```
CREATE TABLE IF NOT EXISTS `conversation_messages` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `message_id` VARCHAR(36) NOT NULL COMMENT '消息UUID',
  `conversation_id` BIGINT UNSIGNED NOT NULL COMMENT '对话ID',
  `parent_message_id` VARCHAR(36) COMMENT '父消息ID',
  `role` VARCHAR(20) NOT NULL COMMENT '角色: user, assistant, system',
  `content` TEXT NOT NULL COMMENT '消息内容',
  `search_query` VARCHAR(500) COMMENT '搜索查询',
  `search_results` JSON COMMENT '搜索结果',
  `search_results_total` INT NOT NULL DEFAULT 0 COMMENT '搜索结果总数',
  `follow_up_questions` TEXT COMMENT '延伸问题',
```

```
  `token_count` INT DEFAULT 0 COMMENT 'Token计数',
  `model` VARCHAR(50) COMMENT 'AI模型标识',
  `finish_reason` VARCHAR(20) COMMENT '完成原因',
  `error` BOOLEAN DEFAULT FALSE COMMENT '是否出错',
  `unfinished` BOOLEAN DEFAULT FALSE COMMENT '是否未完成',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_message_id` (`message_id`),
  INDEX `idx_conversation_id` (`conversation_id`),
  INDEX `idx_parent_message_id` (`parent_message_id`),
  INDEX `idx_conversation_created` (`conversation_id`, `created_at`),
  FOREIGN KEY (`conversation_id`) REFERENCES `conversations`(`id`) ON
DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='对话消息表';
```

4.3.6 订阅计划表 (subscription_plans)

定义系统提供的订阅计划及其配额。

```
CREATE TABLE IF NOT EXISTS `subscription_plans` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `plan_code` VARCHAR(20) NOT NULL COMMENT '计划代码: free, pro, max',
  `plan_name` VARCHAR(50) NOT NULL COMMENT '计划名称',
  `price_monthly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '月费原价',
  `current_price_monthly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '月费
实时价格',
  `price_yearly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '年费原价',
  `current_price_yearly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '年费
实时价格',
  `yearly_discount_enabled` TINYINT NOT NULL DEFAULT 0 COMMENT '年费折扣开
关',
  `yearly_discount_percent` DECIMAL(5,2) NOT NULL DEFAULT 0.00 COMMENT '年
费折扣百分比',
  `features` JSON NOT NULL COMMENT '计划功能和配额',
  `is_active` TINYINT NOT NULL DEFAULT 1 COMMENT '是否启用',
  `sort_order` INT NOT NULL DEFAULT 0 COMMENT '排序顺序',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_plan_code` (`plan_code`),
  INDEX `idx_is_active` (`is_active`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='订阅计划表';
```

4.3.7 使用配额表 (usage_quotas)

记录用户每日的API使用情况。

```
CREATE TABLE IF NOT EXISTS `usage_quotas` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
  `quota_date` DATE NOT NULL COMMENT '使用日期',
  `normal_search_count` INT NOT NULL DEFAULT 0 COMMENT '普通搜索次数',
  `advanced_search_low_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(低)次
数',
  `advanced_search_mid_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(中)次
数',
  `advanced_search_high_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(高)次
数',
  `ai_chat_normal_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(普通)次数',
  `ai_chat_mid_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(中)次数',
  `ai_chat_high_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(高)次数',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_user_date` (`user_id`, `quota_date`),
  INDEX `idx_user_date_desc` (`user_id`, `quota_date` DESC),
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='每日使用配额表';
```

4.3.8 订单表 (orders)

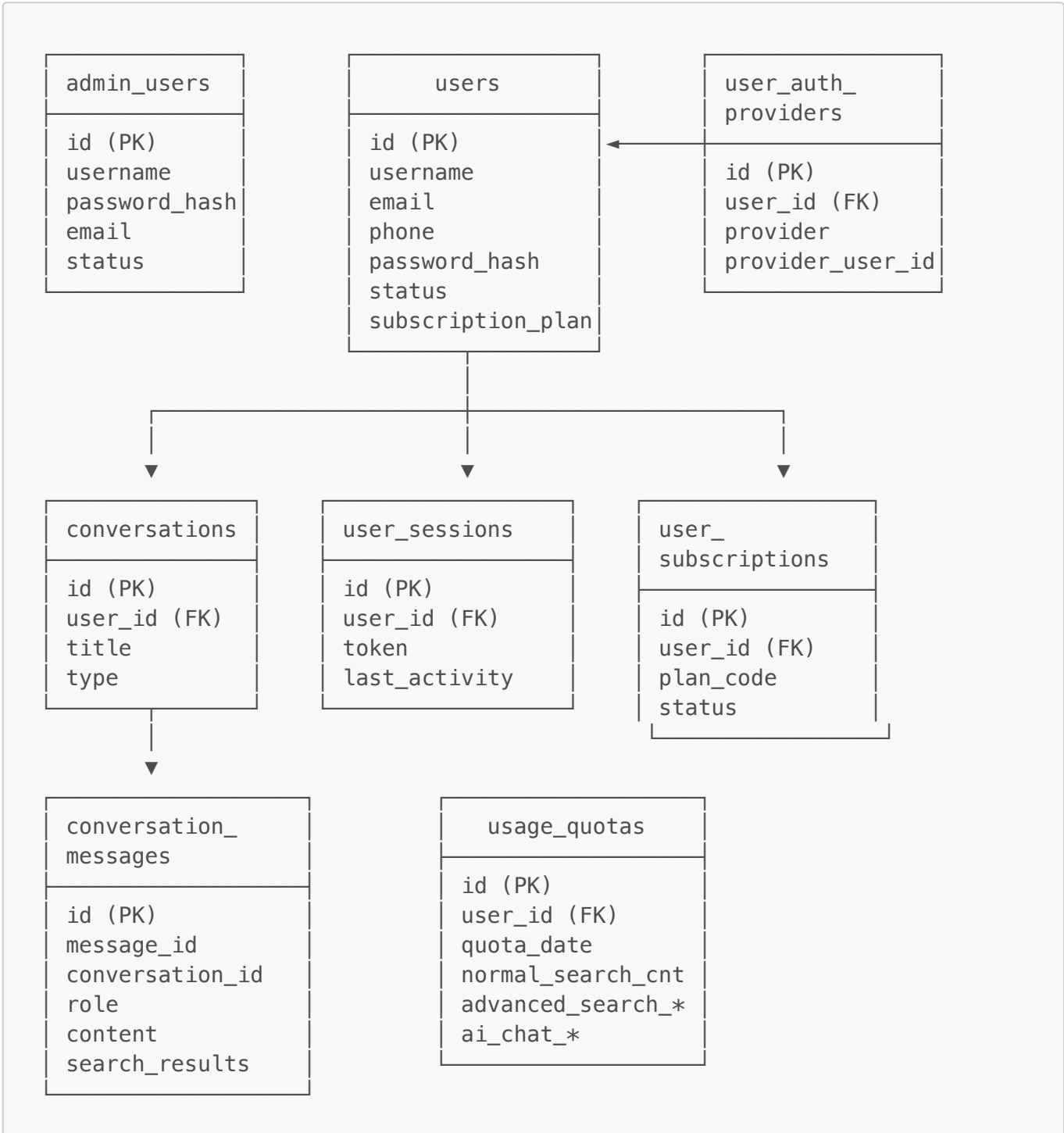
存储用户的订阅订单信息。

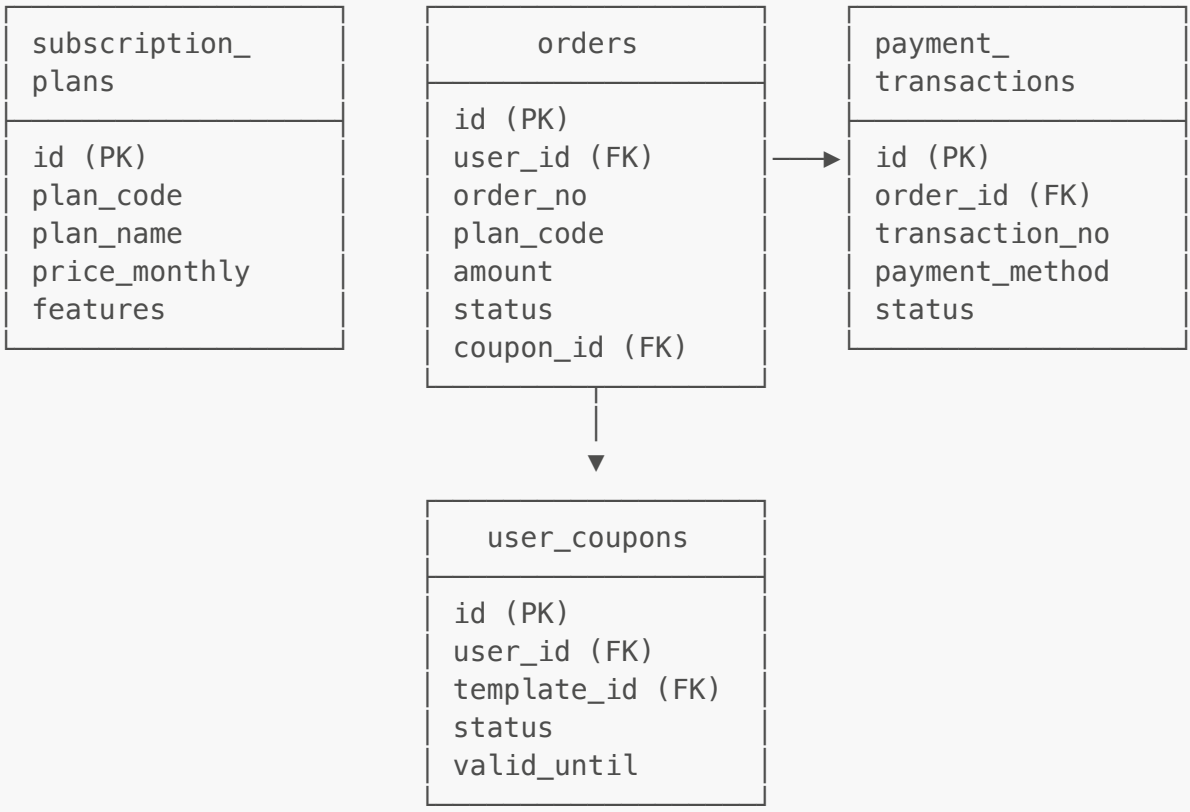
```
CREATE TABLE IF NOT EXISTS `orders` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键ID',
  `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
  `order_no` VARCHAR(64) NOT NULL COMMENT '订单号',
  `plan_code` VARCHAR(20) NOT NULL COMMENT '订阅计划代码',
  `plan_name` VARCHAR(50) NOT NULL COMMENT '计划名称',
  `billing_cycle` VARCHAR(20) NOT NULL COMMENT '计费周期: monthly或yearly',
  `amount` DECIMAL(10,2) NOT NULL COMMENT '订单原价',
  `coupon_id` BIGINT UNSIGNED COMMENT '优惠券ID',
  `discount_amount` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '折扣金额',
  `final_amount` DECIMAL(10,2) NOT NULL COMMENT '最终金额',
  `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT '状态: pending,
paid, cancelled, refunded',
  `payment_method` VARCHAR(50) COMMENT '支付方式',
  `payment_time` DATETIME COMMENT '支付时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时
间',
```

```
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_order_no` (`order_no`),
  INDEX `idx_user_created` (`user_id`, `created_at` DESC),
  INDEX `idx_status_created` (`status`, `created_at` DESC),
  CONSTRAINT `fk_orders_user` FOREIGN KEY (`user_id`) REFERENCES `users`
(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='订单表';
```

4.4 数据库E-R图

以下是系统核心表的实体关系图：





4.5 索引设计

系统采用合理的索引策略以优化查询性能：

主键索引： 所有表均使用BIGINT UNSIGNED自增主键

唯一索引：

- users表：uk_username, uk_email, uk_phone
- orders表：uk_order_no
- conversations表：无（由外键和主键保证）

复合索引（优化常用查询）：

- user_sessions: idx_user_activity (user_id, last_activity DESC)
- conversations: idx_user_type_updated (user_id, conversation_type, updated_at DESC)
- conversation_messages: idx_conversation_created (conversation_id, created_at)
- usage_quotas: idx_user_date_desc (user_id, quota_date DESC)
- orders: idx_user_created (user_id, created_at DESC)

第五章 功能模块设计

5.1 用户管理模块

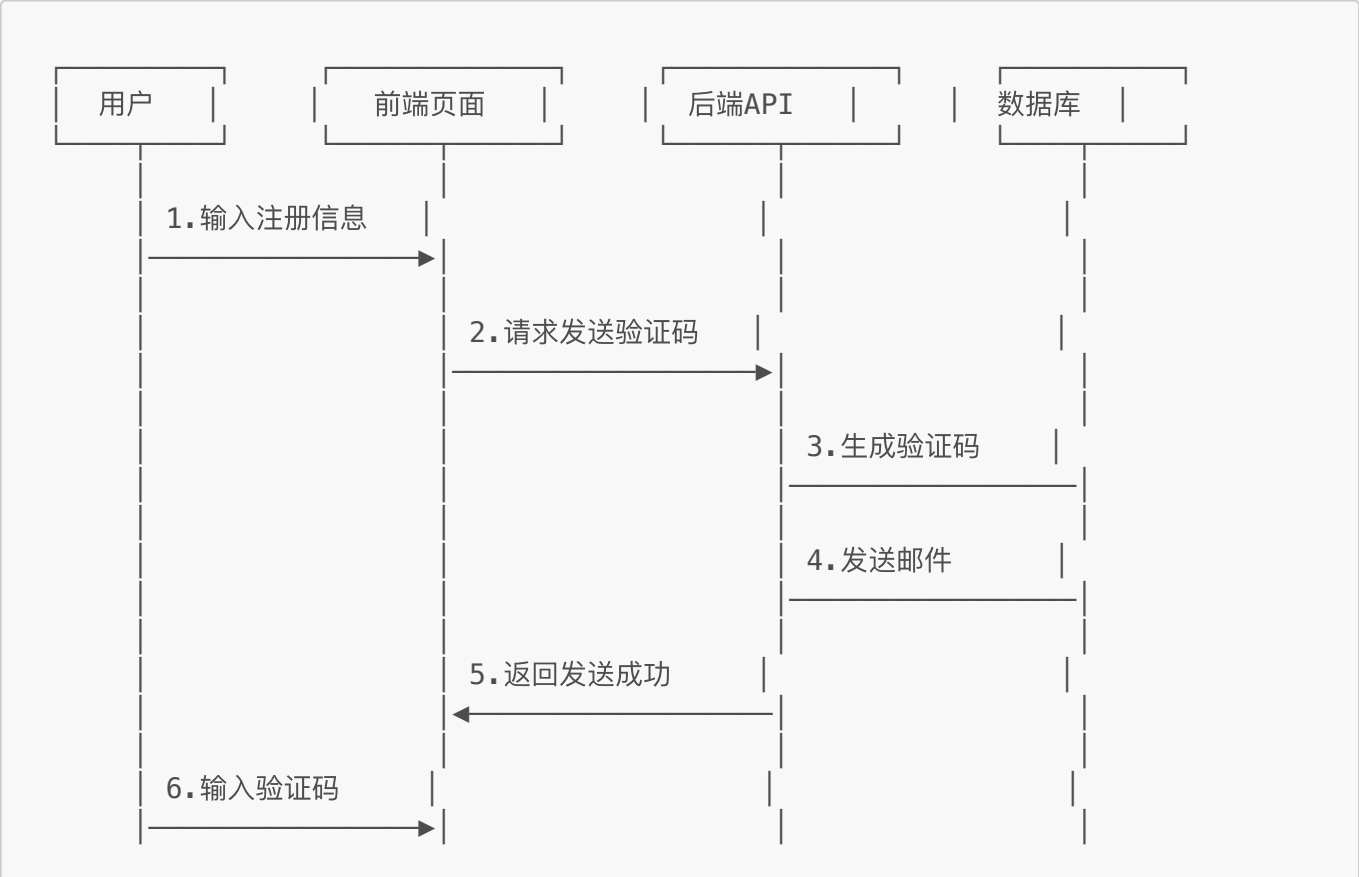
5.1.1 模块概述

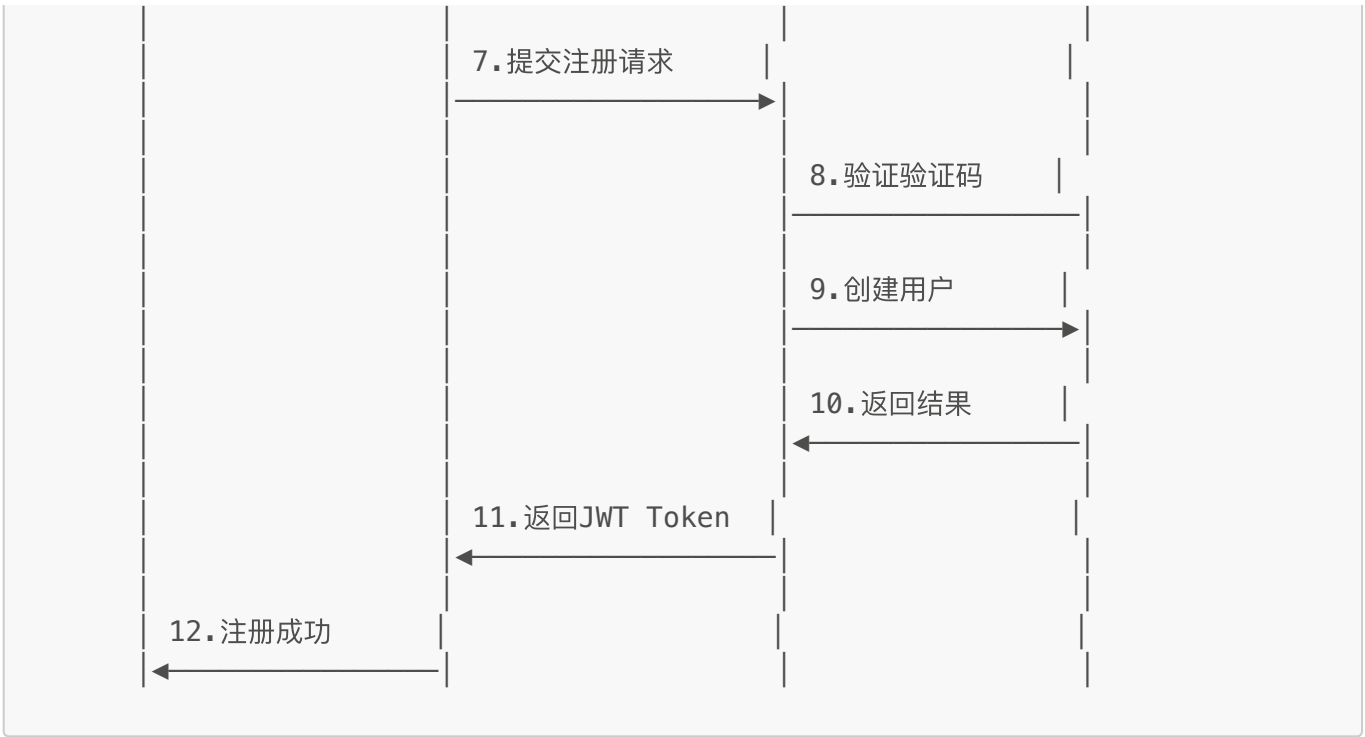
用户管理模块负责处理用户注册、登录、认证、个人信息管理等功能。支持多种认证方式，包括邮箱密码、手机验证码、OAuth（GitHub、Google、微信）等。

5.1.2 功能列表

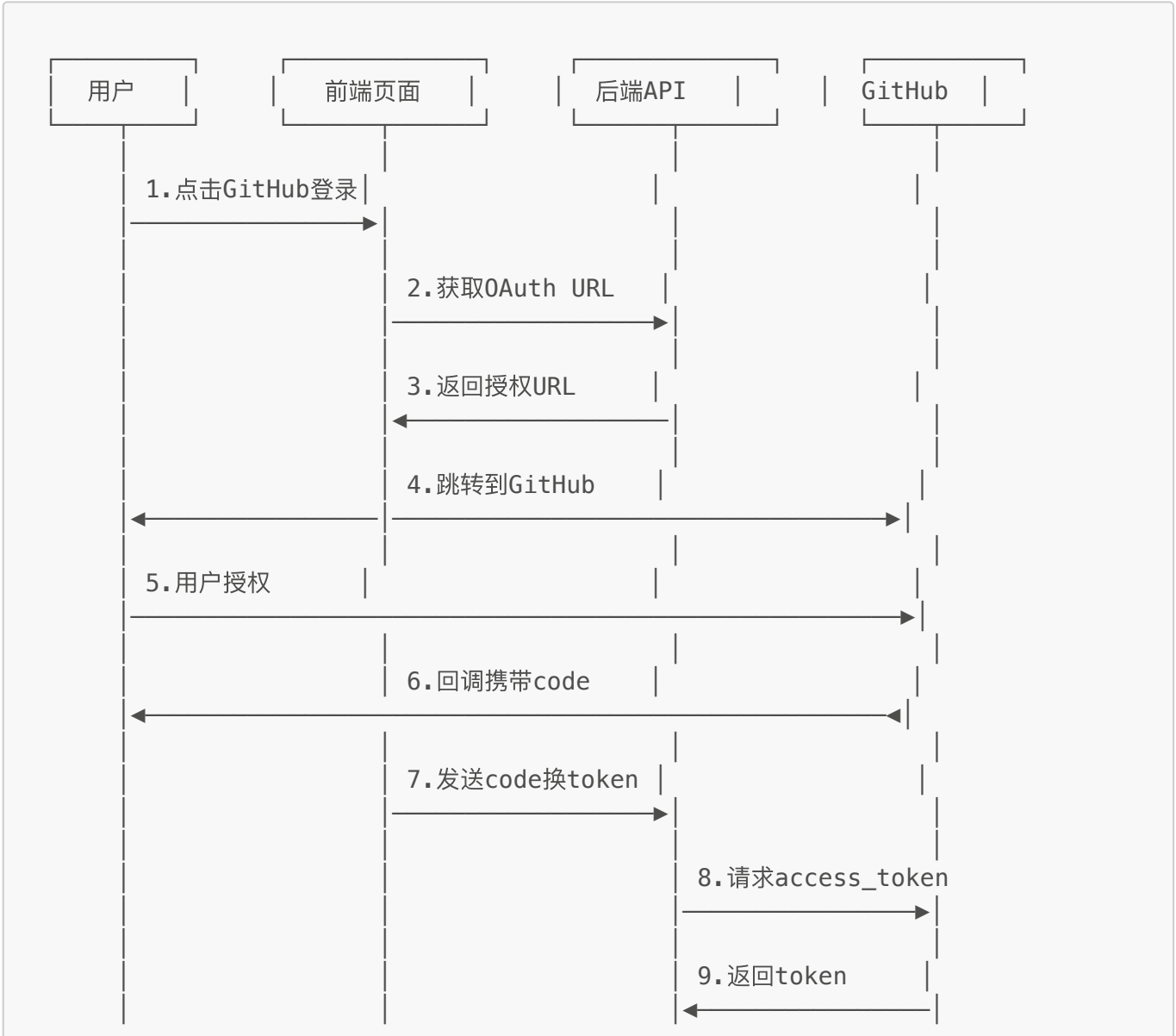
功能编号	功能名称	功能描述
USER-001	邮箱注册	用户通过邮箱和密码注册账户
USER-002	邮箱登录	用户通过邮箱和密码登录系统
USER-003	手机登录	用户通过手机号和验证码登录
USER-004	GitHub登录	用户通过GitHub OAuth登录
USER-005	微信登录	用户通过微信OAuth登录
USER-006	退出登录	用户退出当前登录状态
USER-007	获取个人信息	获取当前登录用户的详细信息
USER-008	更新个人信息	修改用户的基本信息
USER-009	修改密码	用户修改登录密码
USER-010	忘记密码	通过邮箱重置密码
USER-011	绑定手机	为账户绑定手机号
USER-012	绑定邮箱	为账户绑定邮箱

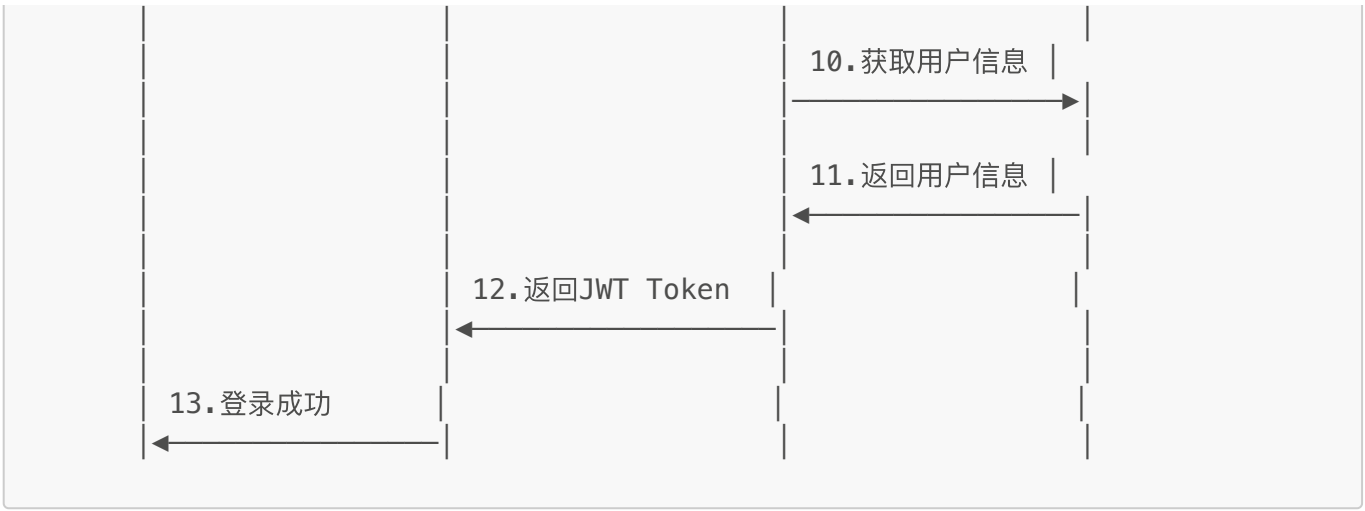
5.1.3 邮箱注册流程





5.1.4 OAuth登录流程





5.2 学术搜索模块

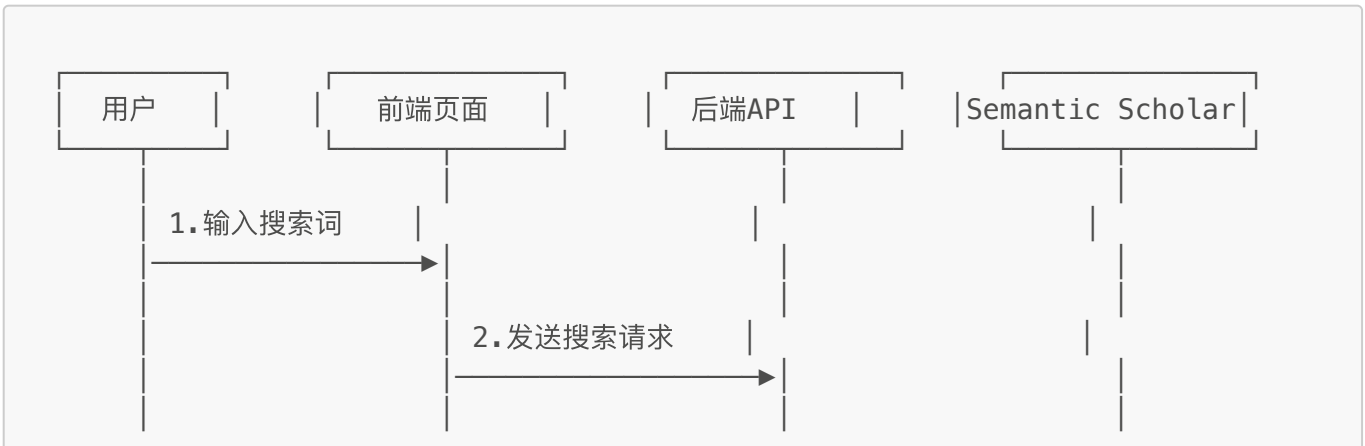
5.2.1 模块概述

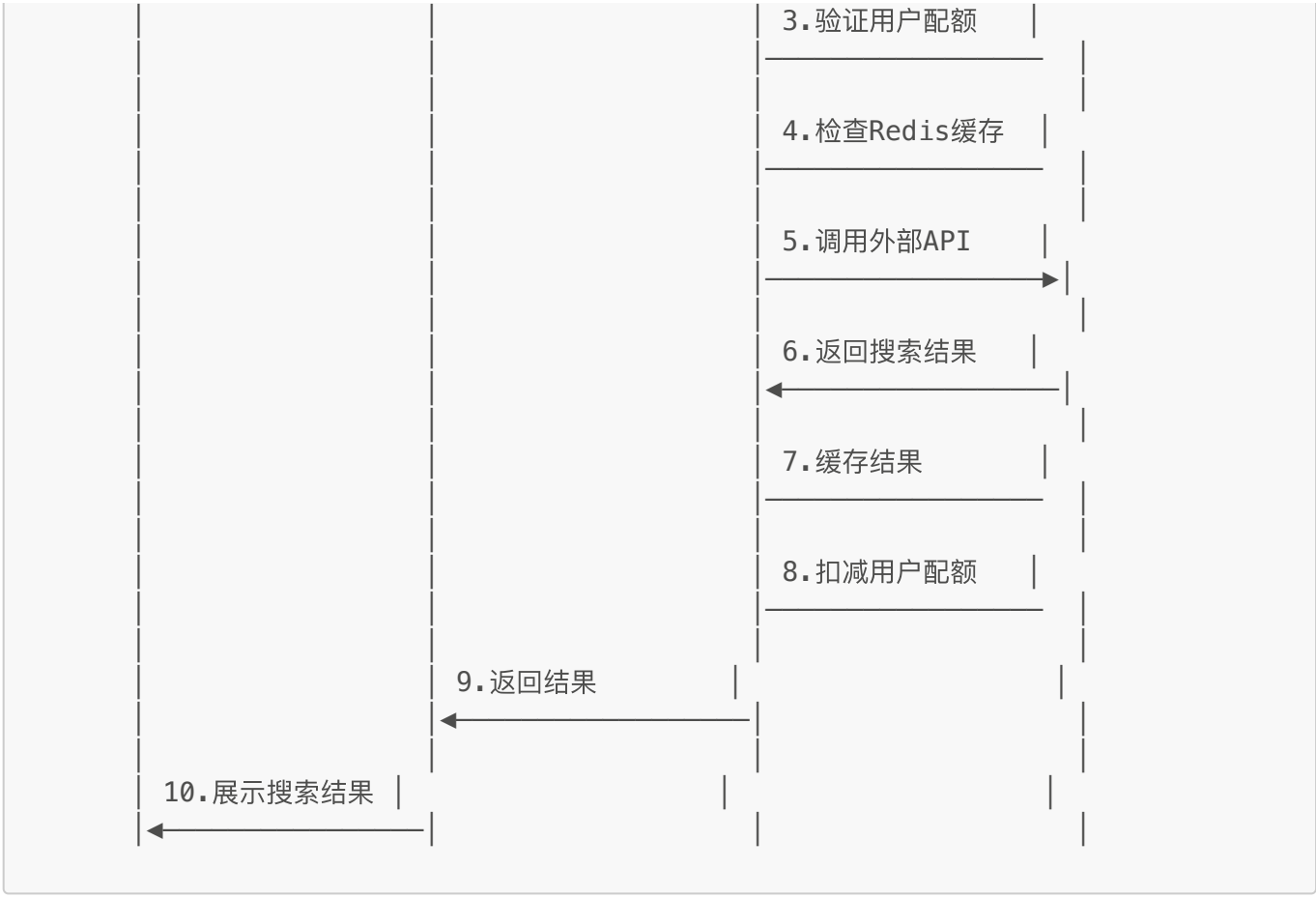
学术搜索模块是系统的核心功能，集成Semantic Scholay学术数据库API，为用户提供论文检索、作者查询、引用分析等功能。

5.2.2 功能列表

功能编号	功能名称	功能描述
SCH-001	论文搜索	根据关键词搜索学术论文
SCH-002	论文详情	获取论文的详细信息
SCH-003	论文引用	获取引用该论文的其他论文列表
SCH-004	论文参考	获取论文引用的参考文献列表
SCH-005	作者搜索	根据姓名搜索学术作者
SCH-006	作者详情	获取作者的详细信息和论文列表
SCH-007	论文推荐	基于论文获取相关推荐
SCH-008	高级搜索	AI辅助的智能语义搜索

5.2.3 搜索流程设计





5.2.4 熔断器设计

为保证系统稳定性，系统为外部API调用配置了熔断器：

```
type CircuitBreaker struct {
    Name          string        // 熔断器名称
    MaxRequests   uint32        // 半开状态最大请求数
    Interval      time.Duration // 统计周期
    Timeout       time.Duration // 熔断超时时间
    ReadyToTrip   func(counts Counts) bool // 熔断判断函数
    OnStateChange func(name string, from State, to State) // 状态变更回调
}
```

熔断策略：

- 连续5次请求失败后触发熔断
- 熔断持续30秒后进入半开状态
- 半开状态允许3次请求测试
- 测试成功则恢复，失败则继续熔断

5.3 AI对话模块

5.3.1 模块概述

AI对话模块提供智能问答功能，支持多轮对话、流式响应、上下文理解等特性。系统支持多种AI模型配置，可根据用户订阅级别使用不同质量的模型。

5.3.2 功能列表

功能编号	功能名称	功能描述
AI-001	创建对话	创建新的AI对话会话
AI-002	发送消息	向对话发送用户消息并获取AI回复
AI-003	流式响应	使用SSE实现实时流式响应
AI-004	对话历史	获取对话的历史消息列表
AI-005	删除对话	删除指定的对话会话
AI-006	论文分析	AI分析指定论文的内容
AI-007	智能搜索	AI辅助的多步骤智能搜索

5.3.3 流式响应实现

系统使用Server-Sent Events (SSE)实现AI响应的流式传输：

```
// SSE响应格式
type StreamResponse struct {
    ID        string `json:"id"`         // 消息ID
    Event     string `json:"event"`      // 事件类型
    Data      string `json:"data"`       // 数据内容
    CreatedAt int64  `json:"created_at"` // 创建时间
}

// 事件类型
const (
    EventStart    = "start"     // 开始响应
    EventContent  = "content"   // 内容片段
    EventSearch   = "search"    // 搜索结果
    EventDone     = "done"      // 响应完成
    EventError    = "error"     // 发生错误
)
```

流式响应流程：

```
客户端                          服务端
    |                             |
    |  GET /chat/stream           |
    |---------------------------->|
    |                             |
    |  HTTP 200                   |
    |  Content-Type: text/event-stream
```

```
    |◀──────────────────────────|
    |                           |
    |  event: start             |
    |  data: {"id":"xxx"}       |
    |◀──────────────────────────|
    |                           |
    |  event: content           |
    |  data: {"text":"Hello"}   |
    |◀──────────────────────────|
    |                           |
    |  event: content           |
    |  data: {"text":" World"}  |
    |◀──────────────────────────|
    |                           |
    |  ...多个content事件...     |
    |                           |
    |  event: done              |
    |  data: {"finish":"stop"}  |
    |◀──────────────────────────|
    |                           |
```

5.3.4 提示词模板管理

系统支持管理员配置AI提示词模板：

```go
type PromptTemplate struct {
    ID          uint64    `json:"id"`
    Name        string    `json:"name"`          // 模板名称
    Description string    `json:"description"`   // 模板描述
    Category    string    `json:"category"`      // 模板分类
    Content     string    `json:"content"`       // 模板内容
    Variables   []string  `json:"variables"`     // 变量列表
    IsActive    bool      `json:"is_active"`     // 是否启用
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}
```

5.4 订阅管理模块

5.4.1 模块概述

订阅管理模块负责处理用户的订阅计划、配额管理、使用量统计等功能。

5.4.2 功能列表

功能编号	功能名称	功能描述
SUB-001	获取订阅计划	获取所有可用的订阅计划

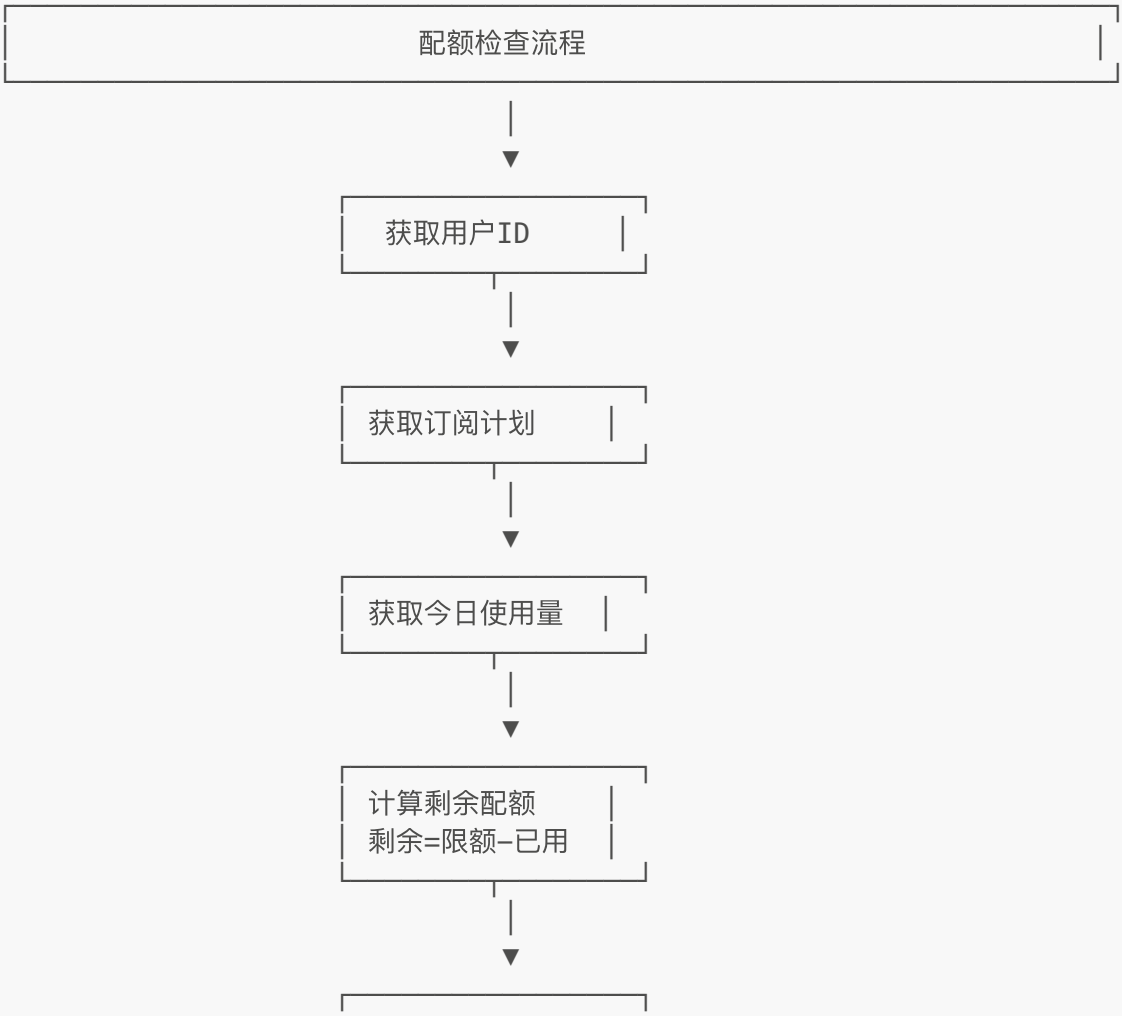
功能编号	功能名称	功能描述
SUB-002	查看当前订阅	获取用户当前的订阅状态
SUB-003	查看配额	获取用户的功能配额信息
SUB-004	查看使用量	获取用户的API使用统计
SUB-005	配额检查	检查用户是否有足够配额
SUB-006	扣减配额	使用功能后扣减相应配额

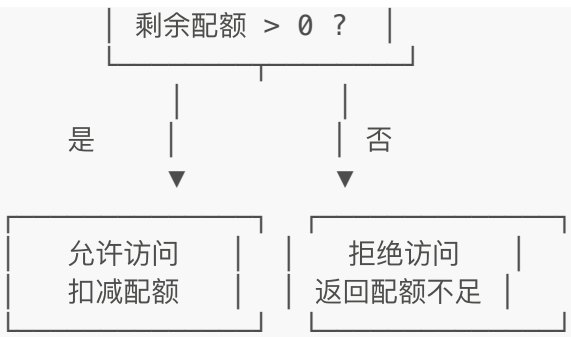
5.4.3 订阅计划配置

系统预设三种订阅计划：

计划	代码	月费	功能配额
免费版	free	¥0	普通搜索50次/天、高级搜索5次/天、AI对话10次/天
专业版	pro	¥19.9	普通搜索无限、高级搜索100次/天、AI对话100次/天
旗舰版	max	¥49.9	所有功能无限使用、优先API响应

5.4.4 配额检查流程





5.5 支付模块

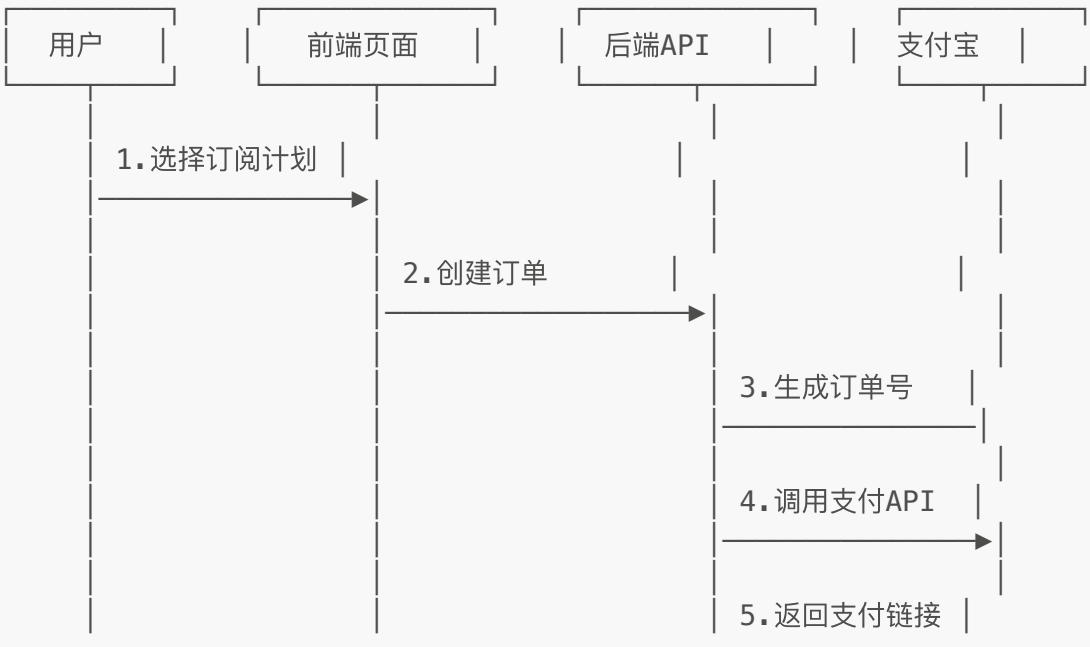
5.5.1 模块概述

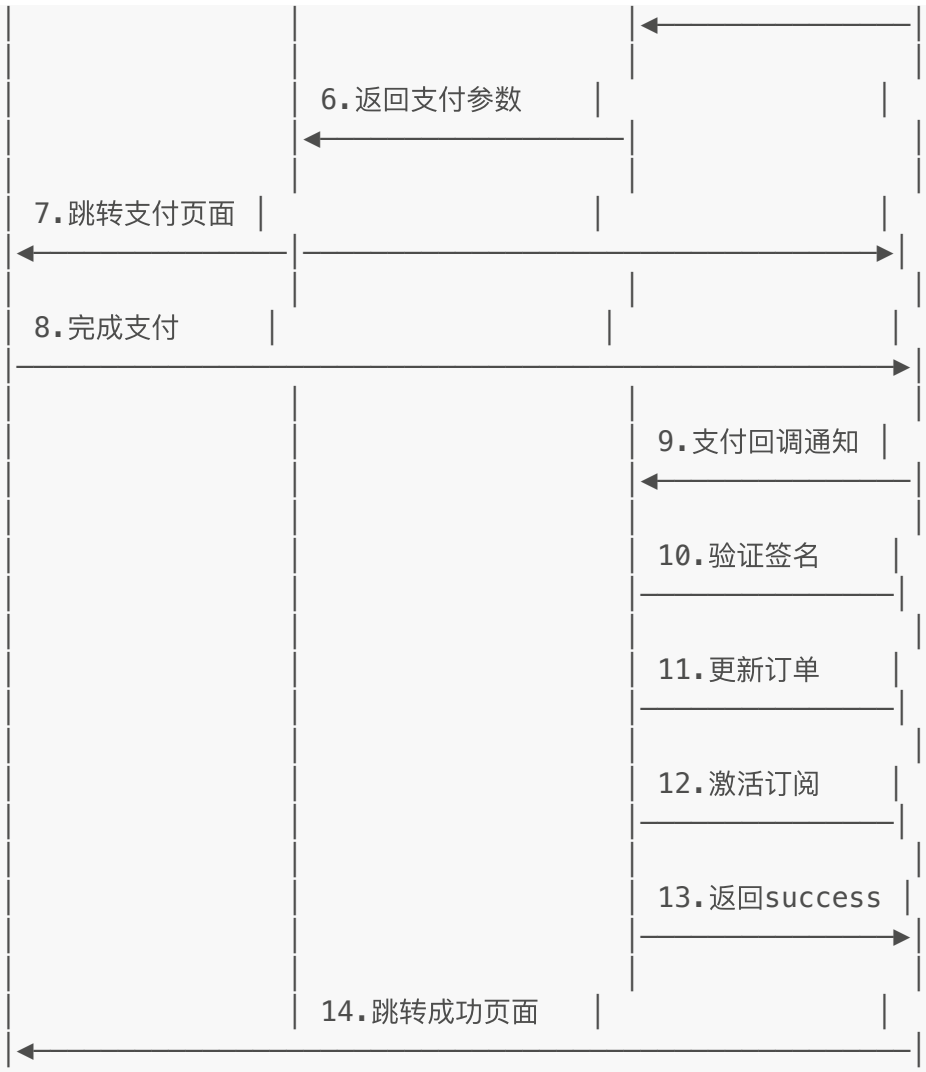
支付模块集成支付宝和微信支付，处理用户的订阅购买订单。

5.5.2 功能列表

功能编号	功能名称	功能描述
PAY-001	创建订单	创建订阅购买订单
PAY-002	支付宝支付	调起支付宝支付
PAY-003	微信支付	调起微信支付
PAY-004	支付回调	处理支付结果通知
PAY-005	订单查询	查询订单状态
PAY-006	申请退款	用户申请订单退款

5.5.3 支付流程设计





5.6 优惠券模块

5.6.1 模块概述

优惠券模块提供灵活的折扣券功能，支持多种折扣类型、发放方式和使用规则。

5.6.2 功能列表

功能编号	功能名称	功能描述
CPN-001	创建优惠券模板	管理员创建优惠券模板
CPN-002	发放优惠券	向用户发放优惠券
CPN-003	领取优惠券	用户领取公开优惠券
CPN-004	我的优惠券	获取用户的优惠券列表
CPN-005	使用优惠券	在订单中使用优惠券
CPN-006	邀请奖励	邀请新用户获得优惠券

5.6.3 优惠券类型

系统支持以下优惠券类型：

类型	代码	说明	示例
百分比折扣	percentage	按订单金额百分比减免	8折优惠券
固定金额	fixed	减免固定金额	满100减20
直减	price_off	直接减少指定金额	立减10元

第六章 接口设计

6.1 接口设计原则

系统API设计遵循RESTful规范，主要原则如下：

1. **资源导向**：以资源为中心设计URL
2. **HTTP方法语义**：GET查询、POST创建、PUT更新、DELETE删除
3. **状态码规范**：使用标准HTTP状态码表示结果
4. **统一响应格式**：所有接口返回统一的JSON格式
5. **版本控制**：URL中包含API版本号

6.2 统一响应格式

成功响应：

```
{
    "code": 0,
    "message": "success",
    "data": {
        // 具体业务数据
    }
}
```

失败响应：

```
{
    "code": 40001,
    "message": "参数错误",
    "data": null
}
```

分页响应：

```
{
    "code": 0,
    "message": "success",
```

```
    "data": {
        "list": [],
        "total": 100,
        "page": 1,
        "page_size": 20
    }
}
```

6.3 错误码定义

错误码	说明
0	成功
40001	参数错误
40002	验证码错误
40003	用户已存在
40004	用户不存在
40005	密码错误
40006	配额不足
40101	未登录
40102	Token过期
40103	权限不足
50001	服务器内部错误
50002	数据库错误
50003	外部服务错误

6.4 核心接口列表

6.4.1 用户接口

方法	路径	说明	认证
POST	/api/v1/user/register	用户注册	否
POST	/api/v1/user/login	用户登录	否
POST	/api/v1/user/logout	用户退出	是
GET	/api/v1/user/profile	获取个人信息	是
PUT	/api/v1/user/profile	更新个人信息	是
POST	/api/v1/user/send-code	发送验证码	否

方法	路径	说明	认证
POST	/api/v1/user/change-password	修改密码	是
GET	/api/v1/user/github/login-url	获取GitHub登录URL	否
POST	/api/v1/user/github/callback	GitHub登录回调	否

6.4.2 学术搜索接口

方法	路径	说明	认证
GET	/api/v1/scholar/papers/search	搜索论文	是
GET	/api/v1/scholar/papers/:id	获取论文详情	是
GET	/api/v1/scholar/papers/:id/citations	获取引用列表	是
GET	/api/v1/scholar/papers/:id/references	获取参考文献	是
GET	/api/v1/scholar/authors/search	搜索作者	是
GET	/api/v1/scholar/authors/:id	获取作者详情	是
GET	/api/v1/scholar/recommendations	获取推荐论文	是

6.4.3 对话接口

方法	路径	说明	认证
GET	/api/v1/user/conversations	获取对话列表	是
POST	/api/v1/user/conversations	创建对话	是
GET	/api/v1/user/conversations/:id	获取对话详情	是
DELETE	/api/v1/user/conversations/:id	删除对话	是
GET	/api/v1/user/conversations/:id/history	获取对话历史	是
POST	/api/v1/user/conversations/:id/messages	发送消息	是
GET	/api/v1/user/conversations/:id/stream	流式响应	是

6.4.4 订阅接口

方法	路径	说明	认证
GET	/api/v1/user/subscription/plans	获取订阅计划	否
GET	/api/v1/user/subscription/current	获取当前订阅	是
GET	/api/v1/user/subscription/quota	获取配额信息	是
GET	/api/v1/user/subscription/usage	获取使用统计	是

6.4.5 订单接口

方法	路径	说明	认证
POST	/api/v1/user/orders/create	创建订单	是
GET	/api/v1/user/orders	获取订单列表	是
GET	/api/v1/user/orders/:id	获取订单详情	是
POST	/api/v1/user/orders/:id/cancel	取消订单	是

6.4.6 管理员接口

方法	路径	说明	认证
POST	/api/v1/admin/login	管理员登录	否
GET	/api/v1/admin/users	获取用户列表	是
GET	/api/v1/admin/users/statistics	获取用户统计	是
GET	/api/v1/admin/orders	获取订单列表	是
GET	/api/v1/admin/orders/statistics	获取订单统计	是
GET	/api/v1/admin/system-config	获取系统配置	是
PUT	/api/v1/admin/system-config	更新系统配置	是

6.5 接口详细说明

6.5.1 用户注册接口

请求：

```
POST /api/v1/user/register
Content-Type: application/json

{
    "email": "user@example.com",
    "password": "password123",
    "verification_code": "123456"
}
```

响应：

```
{
    "code": 0,
    "message": "success",
    "data": {
```

```
        "user": {
            "id": 1,
            "email": "user@example.com",
            "username": null,
            "subscription_plan": "free",
            "created_at": "2025-01-01T00:00:00Z"
        },
        "token": "eyJhbGciOiJIUzI1NiIs..."
    }
}
```

6.5.2 论文搜索接口

请求：

```
GET /api/v1/scholar/papers/search?query=machine+learning&limit=10&offset=0
Authorization: Bearer <token>
```

响应：

```
{
    "code": 0,
    "message": "success",
    "data": {
        "total": 1000000,
        "offset": 0,
        "papers": [
            {
                "paperId": "649def34f8be52c8b66281af98ae884c09aef38b",
                "title": "Deep Learning",
                "abstract": "Deep learning allows computational
models...",
                "year": 2015,
                "citationCount": 42000,
                "authors": [
                    {
                        "authorId": "1847026",
                        "name": "Y. LeCun"
                    }
                ],
                "venue": "Nature",
                "fieldsOfStudy": ["Computer Science"],
                "openAccessPdf": {
                    "url": "https://..."
                }
            }
        ]
    }
}
```

6.5.3 发送消息接口（流式）

请求：

```
GET /api/v1/user/conversations/1/stream?message=请分析这篇论文
Authorization: Bearer <token>
Accept: text/event-stream
```

响应：

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive

event: start
data: {"id":"msg_123","conversation_id":1}

event: content
data: {"text":"这篇论文主要"}

event: content
data: {"text":"研究了深度学习"}

event: content
data: {"text":"在图像识别领域的应用..."}

event: done
data: {"finish_reason":"stop","token_count":256}
```

第七章 安全设计

7.1 认证机制

7.1.1 JWT认证

系统使用JWT（JSON Web Token）进行用户认证：

Token结构：

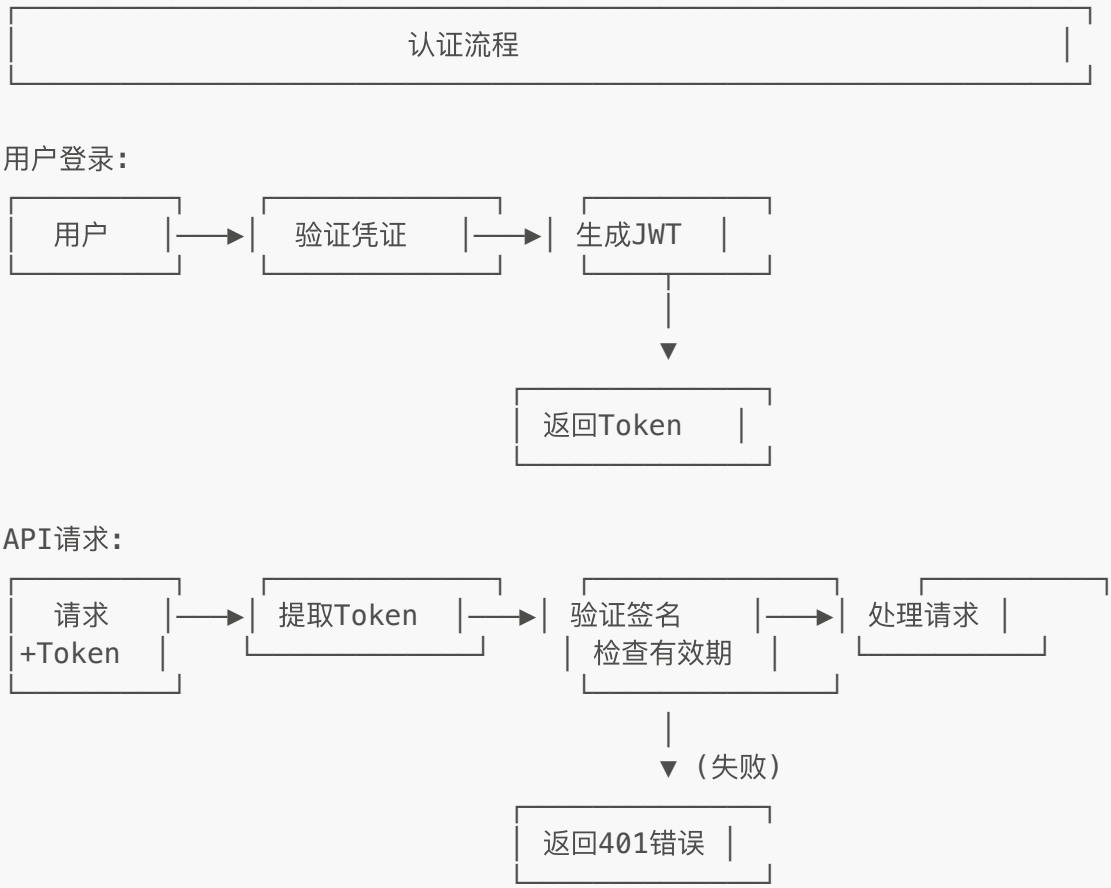
```
{
    "header": {
        "alg": "HS256",
        "typ": "JWT"
    },
```

```
    "payload": {
        "user_id": 1,
        "username": "user@example.com",
        "type": "user",
        "exp": 1735689600,
        "nbf": 1735603200,
        "iat": 1735603200
    },
    "signature": "..."
}
```

Token特性：

- 有效期：14天（可配置）
- 签名算法：HS256
- 自动续期：每次请求刷新有效期

7.1.2 认证流程



7.2 授权机制

7.2.1 角色权限

系统定义两种角色：

角色	标识	权限范围
普通用户	user	用户端API
管理员	admin	管理端API

7.2.2 中间件实现

```
// 用户认证中间件
func UserAuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        token := c.GetHeader("Authorization")
        if token == "" {
            c.JSON(401, gin.H{"code": 40101, "message": "未登录"})
            c.Abort()
            return
        }

        // 验证Token
        claims, err := utils.ParseJWT(token)
        if err != nil {
            c.JSON(401, gin.H{"code": 40102, "message": "Token无效"})
            c.Abort()
            return
        }

        // 检查角色
        if claims.Type != "user" {
            c.JSON(403, gin.H{"code": 40103, "message": "权限不足"})
            c.Abort()
            return
        }

        c.Set("user_id", claims.UserID)
        c.Next()
    }
}
```

7.3 数据安全

7.3.1 密码存储

系统使用bcrypt算法对密码进行哈希存储：

```
import "golang.org/x/crypto/bcrypt"

// 密码加密
func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), 14)
    return string(bytes), err
```

```
}

// 密码验证
func CheckPassword(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}
```

7.3.2 敏感数据处理

- API密钥等敏感配置存储在数据库，不在代码中硬编码
- 支付配置使用加密存储
- 日志中不记录敏感信息（密码、Token等）

7.4 防护机制

7.4.1 速率限制

系统实现了滑动窗口速率限制：

```
type RateLimiter struct {
    maxRequests int           // 最大请求数
    window      time.Duration // 时间窗口
    mu          sync.RWMutex
    requests    map[string][]time.Time
}

// 检查是否允许请求
func (rl *RateLimiter) Allow(key string) bool {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    now := time.Now()
    windowStart := now.Add(-rl.window)

    // 清理过期记录
    valid := make([]time.Time, 0)
    for _, t := range rl.requests[key] {
        if t.After(windowStart) {
            valid = append(valid, t)
        }
    }

    if len(valid) >= rl.maxRequests {
        return false
    }

    rl.requests[key] = append(valid, now)
    return true
}
```

7.4.2 请求超时

系统为不同类型的请求配置了超时机制：

请求类型	超时时间
读取操作	15秒
写入操作	30秒
流式响应	5分钟

7.4.3 CORS配置

```
func CORSMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Header("Access-Control-Allow-Origin", "*")
        c.Header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS")
        c.Header("Access-Control-Allow-Headers", "Origin, Content-Type,
Authorization")
        c.Header("Access-Control-Max-Age", "86400")

        if c.Request.Method == "OPTIONS" {
            c.AbortWithStatus(204)
            return
        }

        c.Next()
    }
}
```

7.5 支付安全

7.5.1 签名验证

所有支付回调都进行签名验证：

```
// 支付宝签名验证
func VerifyAlipaySignature(params map[string]string, publicKey string)
bool {
    sign := params["sign"]
    signType := params["sign_type"]

    // 删除签名相关参数
    delete(params, "sign")
    delete(params, "sign_type")

    // 按字母顺序排序并拼接
```

```
    sorted := sortParams(params)
    signContent := strings.Join(sorted, "&")

    // RSA2验证签名
    return rsaVerify(signContent, sign, publicKey, signType)
}
```

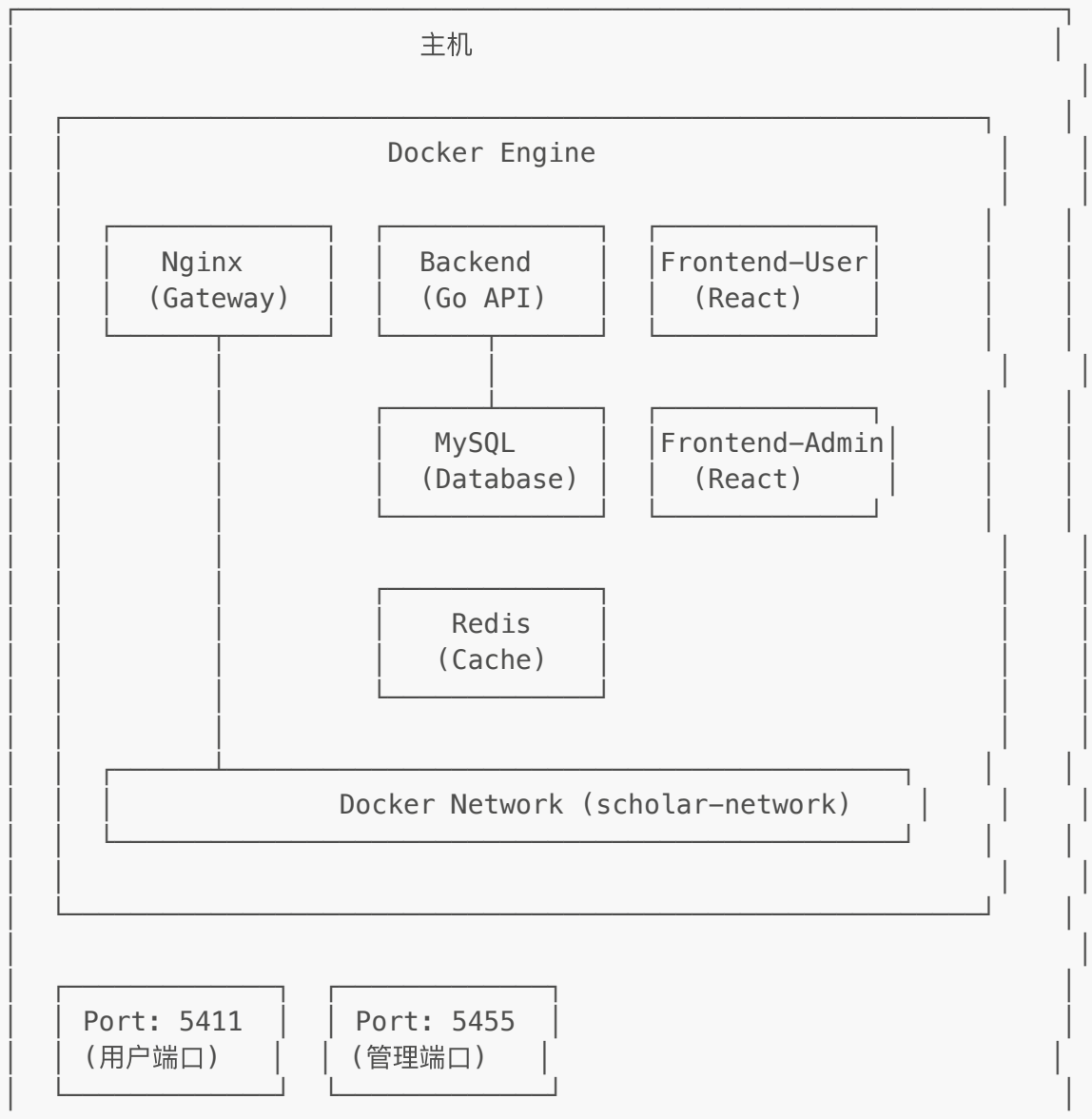
7.5.2 防重放攻击

- 使用订单号作为唯一标识
- 记录已处理的回调通知
- 检查时间戳有效性

第八章 部署设计

8.1 部署架构

系统采用Docker容器化部署，使用Docker Compose进行编排：



```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

8.2 容器配置

8.2.1 服务容器列表

容器名	镜像	端口	说明
scholar_nginx	nginx:alpine	80, 8001	反向代理
scholar_backend	自定义	8080	后端API
scholar_frontend_user	自定义	80	用户前端
scholar_frontend_admin	自定义	80	管理前端
scholar_mysql	mysql:8.0	3306	数据库
scholar_redis	redis:7-alpine	6379	缓存
scholar_schema_sync	自定义	-	数据库同步

8.2.2 Docker Compose配置

```yaml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: scholar_nginx
    ports:
      - "${HTTP_PORT}:80"
      - "${ADMIN_PORT}:8001"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./nginx/conf.d:/etc/nginx/conf.d
    depends_on:
      - backend
      - frontend-user
      - frontend-admin
    networks:
      - scholar-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: scholar_backend
    environment:
      - MYSQL_HOST=mysql
      - MYSQL_PORT=3306
```

```
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
      - REDIS_HOST=redis
      - REDIS_PORT=6379
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_healthy
      schema-sync:
        condition: service_completed_successfully
    networks:
      - scholar-network

  mysql:
    image: mysql:8.0
    container_name: scholar_mysql
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - scholar-network

  redis:
    image: redis:7-alpine
    container_name: scholar_redis
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - scholar-network

networks:
  scholar-network:
    driver: bridge

volumes:
  mysql_data:
  redis_data:
```

8.3 Nginx配置

8.3.1 用户端配置

```
server {
    listen 80;
    server_name localhost;

    # 静态资源
    location / {
        proxy_pass http://frontend-user:80;
        proxy_set_header Host $host;
    }

    # API代理
    location /api/ {
        proxy_pass http://backend:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        # SSE支持
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_buffering off;
        proxy_cache off;
    }

    # 静态资源缓存
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$ {
        proxy_pass http://frontend-user:80;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

8.4 启动流程

系统启动顺序：

```
1. MySQL启动 → 等待健康检查通过
        ↓
2. Redis启动 → 等待健康检查通过
        ↓
3. Schema-sync执行 → 数据库结构同步
        ↓
4. Backend启动 → 连接MySQL和Redis
        ↓
```

```
5. Frontend-user启动
        ↓
6. Frontend-admin启动
        ↓
7. Nginx启动 → 反向代理所有服务
```

8.5 运维命令

```
# 启动所有服务
docker-compose up -d --build

# 停止所有服务
docker-compose down

# 查看服务状态
docker-compose ps

# 查看日志
docker-compose logs -f backend
docker-compose logs -f nginx

# 重启单个服务
docker-compose restart backend

# 重建单个服务
docker-compose up -d --build backend

# 进入容器
docker exec -it scholar_backend sh
docker exec -it scholar_mysql mysql -u scholar_user -p

# 数据库备份
docker exec scholar_mysql mysqldump -u scholar_user -p scholar >
backup.sql
```

第九章 测试说明

9.1 测试策略

系统采用多层次测试策略：

测试类型	测试范围	工具
单元测试	业务逻辑函数	Go testing
集成测试	API接口	Postman/curl
性能测试	系统负载	wrk/ab
安全测试	漏洞扫描	OWASP ZAP

9.2 测试用例示例

9.2.1 用户注册测试

用例ID	测试项	输入	预期结果
TC-001	正常注册	有效邮箱+密码+验证码	返回用户信息和Token
TC-002	邮箱已存在	已注册邮箱	返回40003错误
TC-003	验证码错误	错误的验证码	返回40002错误
TC-004	密码过短	5位密码	返回40001错误

9.2.2 论文搜索测试

用例ID	测试项	输入	预期结果
TC-101	正常搜索	query=machine learning	返回论文列表
TC-102	空查询	query=	返回40001错误
TC-103	分页查询	offset=10&limit=20	返回第2页数据
TC-104	配额不足	超出每日限额	返回40006错误

9.3 性能指标

系统性能目标：

指标	目标值
响应时间（P95）	< 500ms
吞吐量	> 1000 QPS
可用性	> 99.9%
并发用户	> 500

第十章 附录

10.1 项目目录结构

```
scholar/
├── backend/                    # 后端代码
│   ├── cmd/
│   │   └── server/
│   │       └── main.go        # 入口文件
│   ├── internal/
│   │   ├── admin/             # 管理员模块
│   │   ├── user/              # 用户模块
│   │   ├── ai/                # AI服务模块
```

```
│   │   ├── conversation/       # 对话模块
│   │   ├── semanticscholar/    # 学术搜索模块
│   │   ├── subscription/       # 订阅模块
│   │   ├── order/              # 订单模块
│   │   ├── payment/            # 支付模块
│   │   ├── coupon/             # 优惠券模块
│   │   ├── config/             # 配置模块
│   │   ├── proxy/              # 代理模块
│   │   ├── mcp/                # MCP协议模块
│   │   └── common/             # 公共模块
│   ├── Dockerfile
│   └── go.mod
├── frontend-user/               # 用户前端
│   ├── src/
│   │   ├── components/         # 组件
│   │   ├── pages/              # 页面
│   │   ├── services/           # API服务
│   │   ├── contexts/           # 上下文
│   │   └── App.js
│   ├── Dockerfile
│   └── package.json
├── frontend-admin/              # 管理前端
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.js
│   ├── Dockerfile
│   └── package.json
├── docker/
│   └── mysql/
│       └── init/
│           └── 01_schema.sql  # 数据库初始化
├── nginx/
│   ├── nginx.conf              # Nginx主配置
│   └── conf.d/
│       ├── user.conf           # 用户端配置
│       └── admin.conf          # 管理端配置
├── scripts/                     # 运维脚本
├── docker-compose.yml           # 容器编排
├── .env                         # 环境变量
└── CLAUDE.md                    # 项目文档
```

10.2 环境变量说明

变量名	说明	默认值
HTTP_PORT	用户端口	5411
ADMIN_PORT	管理端口	5455
MYSQL_DATABASE	数据库名	scholar

变量名	说明	默认值
MYSQL_USER	数据库用户	scholar_user
MYSQL_PASSWORD	数据库密码	scholar_pass
MYSQL_ROOT_PASSWORD	Root密码	root_password
JWT_SECRET	JWT密钥	your-secret-key
REDIS_PASSWORD	Redis密码	（空）

10.3 版本历史

版本	日期	说明
V1.0	2025-01	首次发布

10.4 术语表

术语	定义
论文	学术研究文献，包含标题、摘要、作者等信息
引用	一篇论文引用另一篇论文作为参考
对话	用户与AI系统的一次交互会话
订阅	用户购买的服务计划
配额	用户在一定时间内可使用的功能次数限制
优惠券	用于订单折扣的电子凭证

文档结束

版权信息

软件名称： Scholay学术

版本号： V1.0

著作权人： 成都恩特罐罐文化科技有限公司

统一社会信用代码： 91510100MAEEHRNE26

地址： 中国四川省成都市

Copyright © 2025 成都恩特罐罐文化科技有限公司

All Rights Reserved. 保留所有权利。

未经著作权人书面许可，任何单位和个人不得擅自复制、修改、发行本软件及其文档。

编制单位：成都恩特罐罐文化科技有限公司

编制日期：2025年1月